



Recitation 2 (part 2/2)

Redirection & Piping

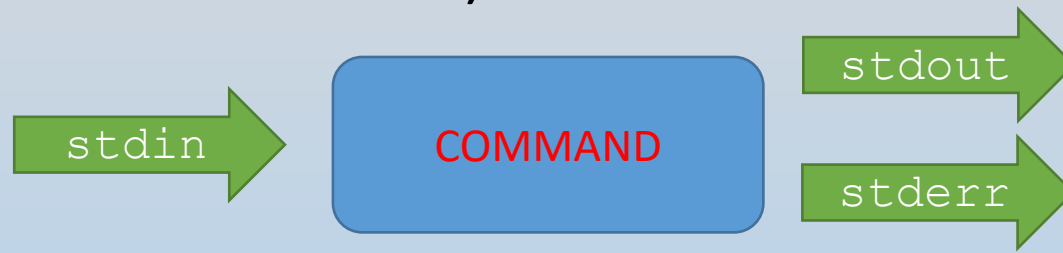


Recitation overview

- Review of redirection and piping
- Parsing delimited tables – exercises 1-3 (Problem 1 from HW 2)
- Word counting (Problem 2 from HW 2)

Input and output redirection

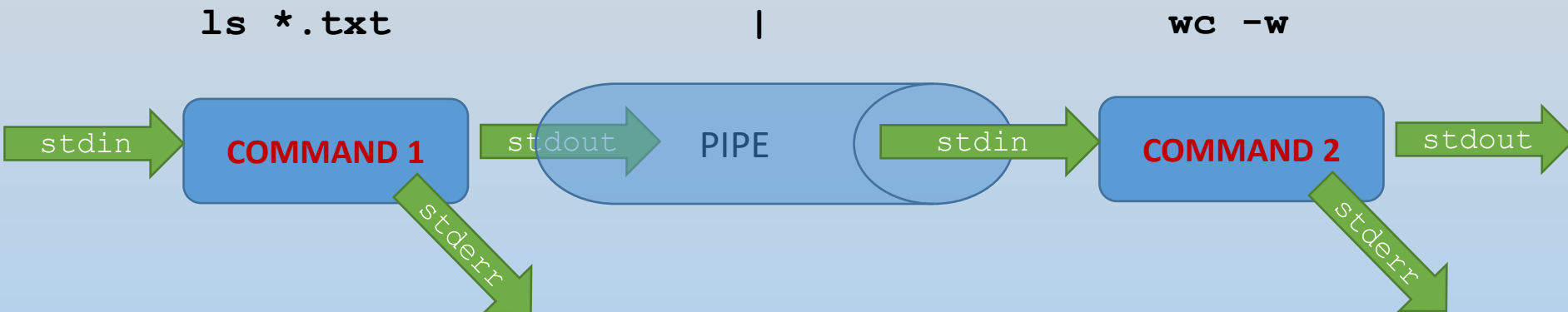
- Normally, when you run a command, it gets input from the keyboard (**stdin**) and sends output to the screen (**stdout** and **stderr**)



- Use '**>**' or '**1>**' to redirect **stdout** (standard output) to file
- Use '**2>**' to redirect **stderr** (standard error) to file
- Use '**&>**' to redirect **stdout** and **stderr** to file together
- Use '**<**' to redirect file into **stdin** (standard input) of command

Pipes

A *pipe* takes the output of one command, and passes it as input to another



Notes for the exercises

- All the input files, unless specified otherwise, can be found in `/share/recitations/recitation02/`
- These are the same files you worked with in the lecture.
- For convenience, you should create a local directory in your homedir, and copy all of the files from `recitation02` into it before starting.
- If you get stuck – the slide the exercise presentation will contain hints.

Exercise 1

Write a piped sequence of commands that takes the file `phone_dir.txt` and generates a file `phone_reformatted.txt`, in which each line is formatted as follows:

<First_name>`<tab>`**<Phone_num>** and the first name is in all-caps.

First 3 lines of expected output:

```
ANDREW  7583
BRUCE   6466
RYAN    6585
```

Exercise 1 - planning

“Plans are worthless, but planning is everything.” – Dwight Eisenhower

1. What is the input?
2. What is the required output?
3. Can you think of a single command that gets the input closer to the output?
4. Run it! This is your new input.
5. Go back to 3, until you’ve reached the required output.

Exercise 1 – input & output

First 3 lines of input:

```
ADAMS, Andrew 7583  
BARRETT, Bruce 6466  
BAYES, Ryan 6585
```

First 3 lines of expected output:

```
ANDREW      7583  
BRUCE       6466  
RYAN        6585
```

1. Change spaces to tabs
2. Select appropriate fields (2 [first name] and 3 [phone number])
3. Change the case of the text to uppercase
4. Write output to the appropriate file

Exercise 1 - solution

1. Change spaces to tabs

```
cat phone_dir.txt | tr " " "\t"
```

2. Select fields 2 and 3

```
cat phone_dir.txt | tr " " "\t" | \  
cut -f2-3
```

3. Change the case of the text to uppercase

```
cat phone_dir.txt | tr " " "\t" | \  
cut -f2-3 | tr a-z A-Z
```

4. Write output to appropriate file

```
cat phone_dir.txt | tr " " "\t" | \  
cut -f2-3 | tr a-z A-Z > phone_reformatted.txt
```

Exercise 2

Write a piped sequence of commands that takes the file `phone_dir.txt` and generates a file `phone_sorted.txt`, in which the entries are sorted first according to the first name, and for people with the same first name, in reverse order of the phone number.

Exercise 2 – with hints

Write a piped sequence of commands that takes the file `phone_dir.txt` and generates a file `phone_sorted.txt`, in which the entries are sorted first according to the first name, and for people with the same first name, in reverse order of the phone number.

Reminders/hints:

- You don't *have* to use pipes if there's a simpler/more correct solution
- `sort` can be given multiple keys (as sequences of `-k` flags) to sort hierarchically
- Recall the difference between lexicographical and numerical order

Exercise 2 - solution

```
sort -k2,2 -k3,3nr phone_dir.txt > phone_sorted.txt
```

Notes for this solution:

1. We must sort by name and *then* by the phone number; therefore, we can't use two (piped) sort commands, but instead multiple `-k` flags
2. `sort` accepts the file name as input; therefore we don't have to pipe it in via `cat` (but we can if we really want to)
3. We don't have to use `3, 3` for the second `-k`, because there are only 3 columns, but this way is more robust to changes in the input
4. Since we're sorting by phone *numbers*, we should use numerical, not lexicographic, sorting

Exercise 3

Write a piped sequence of commands that takes the file `phone_dir.txt` and generates a file `phone_short.txt`, in which only the 2nd and 3rd digits of the phone number are specified.

In other words, remove the first and last digit in each number

First three lines of expected output:

ADAMS, Andrew 7583	→ ADAMS, Andrew 58
BARRETT, Bruce 6466	→ BARRETT, Bruce 46
BAYES, Ryan 6585	→ BAYES, Ryan 58

Exercise 3 – with hints

Write a piped sequence of commands that takes the file `phone_dir.txt` and generates a file `phone_short.txt`, in which only the 2nd and 3rd digits of the phone number are specified.

In other words, remove the first and last digit in each number

Hints:

1. To get “mixed” output, you can create a large “aggregate” and then select the relevant fields from it
2. Some commands don’t use `stdin` as input, but you can “camouflage” `stdin` as a file bypassing – (a dash) as a parameter

Exercise 3 - plan

First 3 lines of input:

```
ADAMS, Andrew 7583  
BARRETT, Bruce  
6466  
BAYES, Ryan 6585
```

First 3 lines of expected output:

```
ADAMS, Andrew 58  
BARRETT, Bruce 46  
BAYES, Ryan 58
```

1. Cut the third column and transform it.
2. Paste the transformed column onto the original file.
3. Select only the relevant fields.

Exercise 3 - solution

```
cat phone_dir.txt | \  
cut -d" " -f3 | \  
cut -c2,3 | \  
paste phone_dir.txt - -d " " | \  
cut -d" " -f1,2,4 > phone_short.txt
```

Note the order of the inputs given to `paste`.

File processing using pipes – example

Go over the story “A tale of two cities” and print the ten most frequent words in the file, with their number of appearances.

Question: how do you define a word?

Answer 1: a string of non-white-space characters between two white space characters.

<code>cat a-tale-of-two-cities.txt \</code>	(print file)
<code>tr [:space:] "\n" \</code>	(put words in separate lines)
<code>sort \</code>	(sort words)
<code>uniq -c \</code>	(count words – after sorting)
<code>sort -k1,1nr \</code>	(sort based on counts)
<code>head</code>	

File processing using pipes – example

Go over the story “A tale of two cities” and print the ten most frequent words in the file, with their number of appearances.

```
cat a-tale-of-two-cities.txt | \  
tr [:space:] "\n" | \  
sort | \  
uniq -c | \  
sort -k1,1nr | \  
head
```

(print file)
(put words in separate lines)
(sort words)
(count words – after sorting)
(sort based on counts)

Follow-up issues: (in recitation)

- Why is the “empty word” counted and how can we remove it?
- What about case-sensitivity?
- What to do with punctuation marks and digits? Delete? Use as word separators?

The “empty” word

Consider the output of the first two commands:

```
cat a-tale-of-two-cities.txt | tr [:space:] "\n" | less
```

- Empty lines are “words” – words of length zero, empty strings.
- At this part of the course, it’s hard to simply remove empty lines.
- But – since every line ends with the newline character (“\n”), we can “squeeze” the newlines for the same effect:

```
cat a-tale-of-two-cities.txt | \  
tr [:space:] "\n" | tr -s "\n" | \  
less
```

Case sensitivity

- In Linux everything is case sensitive (unless specified otherwise)
- Some commands can ignore case sensitivity
- But if they don't have that option, we can simply "standardize" the text by converting it all to lower or uppercase:

```
cat a-tale-of-two-cities.txt | \  
tr [:space:] "\n" | tr -s "\n" | \  
tr A-Z a-z | \  
less
```

Punctuation marks

"Off with their heads!" – the Queen of Hearts

```
cat a-tale-of-two-cities.txt | \  
tr [:space:] "\n" | tr -s "\n" | \  
tr A-Z a-z | \  
tr -d [:punct:] | \  
less
```

The complete solution

```
cat a-tale-of-two-cities.txt | \  
tr [:space:] "\n" | tr -s "\n" | \  
tr A-Z a-z | \  
tr -d [:punct:] | \  
sort | \  
uniq -c | \  
sort -k1,1nr | \  
head
```